D-05. 상세 설계서 — VOD 서비스 (Detailed Design)

> ***문서 정보***

항목	내용
프로젝트명	2026_TV — VOD 서비스
문서 번호	D-05 (VOD)
문서 버전	v1.0
작성일	2026-03-04
포함 범위	**슬롯 큐레이션 · NLP 추천 · FAST 광고 파이프라인 · AdOverlay**

---

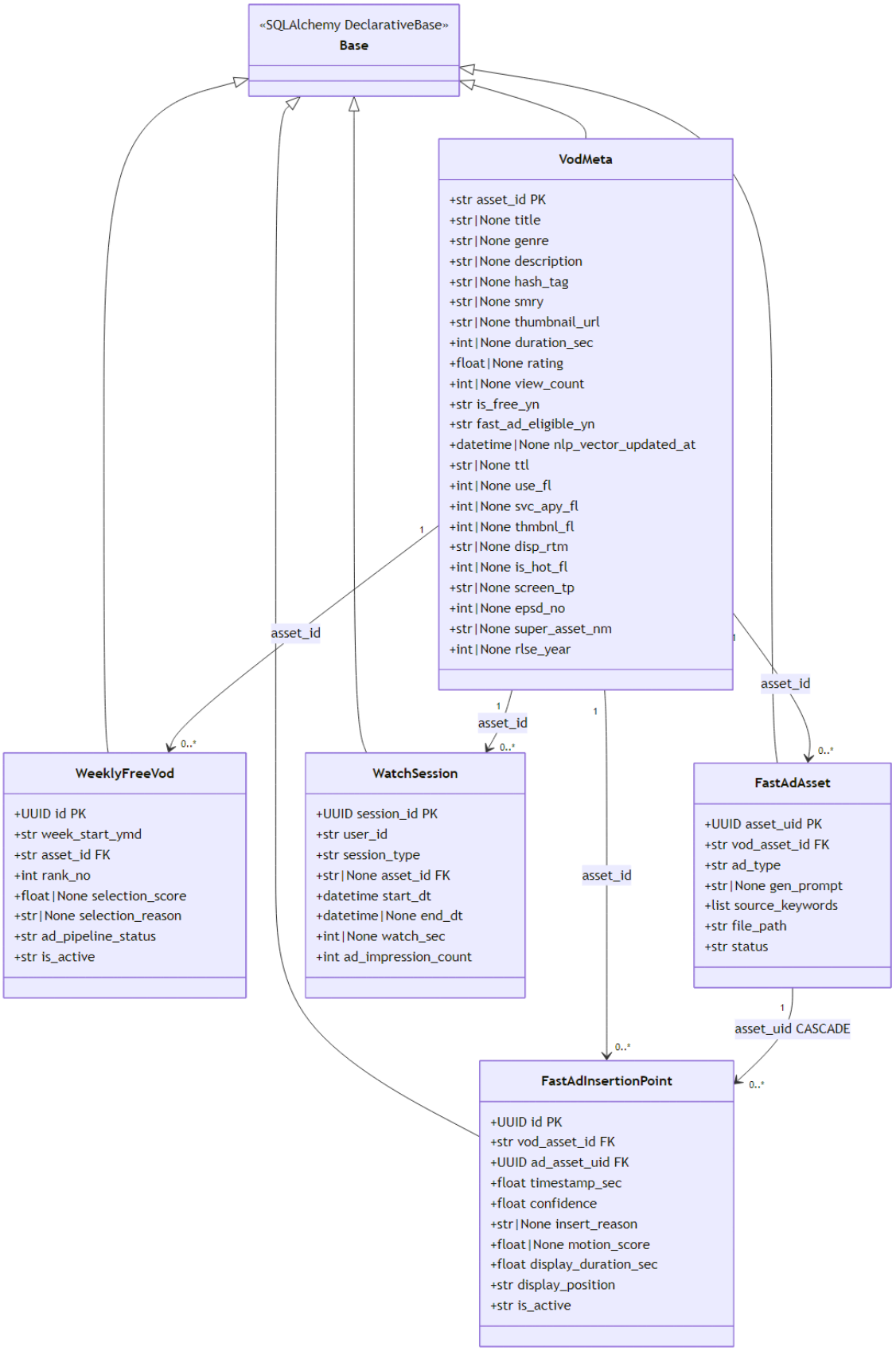
1. 클래스 다이어그램

1.1 backend-api — VOD 관련 ORM 모델

```
classDiagram
    class Base {
        <<SQLAlchemy DeclarativeBase>>
    }

    class VodMeta {
        +str asset_id  PK
        +str|None title
        +str|None genre
        +str|None description
        +str|None hash_tag
        +str|None smry
        +str|None thumbnail_url
        +int|None duration_sec
        +float|None rating
        +int|None view_count
        +str is_free_yn
        +str fast_ad_eligible_yn
        +datetime|None nlp_vector_updated_at
        +str|None ttl
        +int|None use_fl
        +int|None svc_apy_fl
        +int|None thmbnl_fl
        +str|None disp_rtm
        +int|None is_hot_fl
        +str|None screen_tp
        +int|None epsd_no
        +str|None super_asset_nm
        +int|None rlse_year
    }
```

```
class WeeklyFreeVod {
    +UUID id  PK
    +str week_start_ymd
    +str asset_id  FK
    +int rank_no
    +float|None selection_score
    +str|None selection_reason
    +str ad_pipeline_status
    +str is_active
}

class FastAdAsset {
    +UUID asset_uid  PK
    +str vod_asset_id  FK
    +str ad_type
    +str|None gen_prompt
    +list source_keywords
    +str file_path
    +str status
}

class FastAdInsertionPoint {
    +UUID id  PK
    +str vod_asset_id  FK
    +UUID ad_asset_uid  FK
    +float timestamp_sec
    +float confidence
    +str|None insert_reason
    +float|None motion_score
    +float display_duration_sec
    +str display_position
    +str is_active
}

class WatchSession {
    +UUID session_id  PK
    +str user_id
    +str session_type
    +str|None asset_id  FK
    +datetime start_dt
    +datetime|None end_dt
    +int|None watch_sec
    +int ad_impression_count
}

Base <|-- VodMeta
Base <|-- WeeklyFreeVod
Base <|-- FastAdAsset
Base <|-- FastAdInsertionPoint
Base <|-- WatchSession

VodMeta "1" --> "0..*" WeeklyFreeVod : asset_id
```

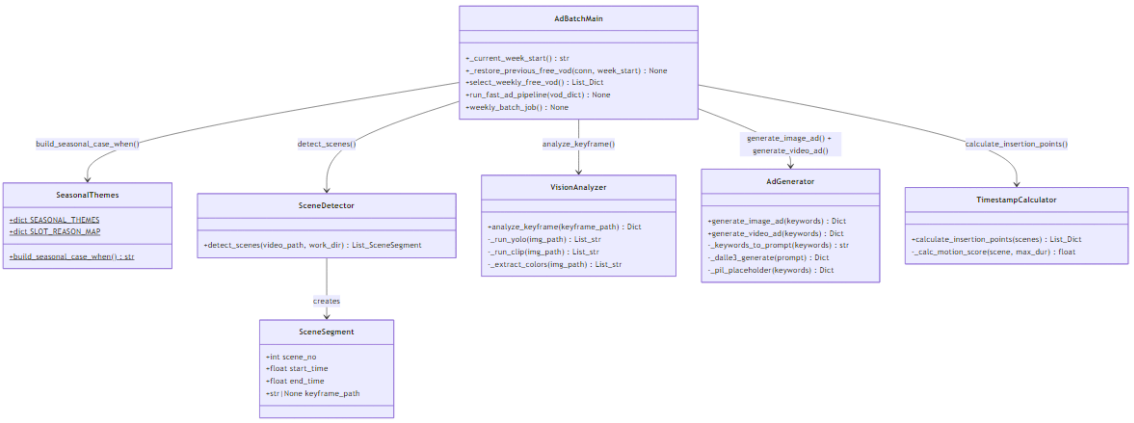
```
VodMeta "1" --> "0..*" FastAdAsset : asset_id
VodMeta "1" --> "0..*" FastAdInsertionPoint : asset_id
VodMeta "1" --> "0..*" WatchSession : asset_id
FastAdAsset "1" --> "0..*" FastAdInsertionPoint : asset_uid CASCADE
```


«SQLAlchemy DeclarativeBase»
Base
VodMeta
+str asset_id PK
+str|None title
+str|None genre
+str|None description
+str|None hash_tag
+str|None smry
+str|None thumbnail_url
+int|None duration_sec
+float|None rating
+int|None view_count
+str is_free_yn
+str fast_ad_eligible_yn
+datetime|None nlp_vector_updated_at
+str|None ttl
+int|None use_fl
+int|None svc_apy_fl
+int|None thmbnl_fl
+str|None disp_rtm
+int|None is_hot_fl
+str|None screen_tp
+int|None epsd_no
+str|None super_asset_nm
+int|None rlse_year
asset_id
asset_id
asset_id
asset_id
asset_uid CASCADE
1
0..*
WeeklyFreeVod
+UUID id PK
+str week_start_ymd
+str asset_id FK
+int rank_no
+float|None selection_score
+str|None selection_reason
+str ad_pipeline_status
+str is_active
WatchSession
+UUID session_id PK
+str user_id
+str session_type
+str|None asset_id FK
+datetime start_dt
+datetime|None end_dt
+int|None watch_sec
+int ad_impression_count
FastAdAsset
+UUID asset_uid PK
+str vod_asset_id FK
+str ad_type
+str|None gen_prompt
+list source_keywords
+str file_path
+str status
FastAdInsertionPoint
+UUID id PK
+str vod_asset_id FK
+UUID ad_asset_uid FK
+float timestamp_sec
+float confidence
+str|None insert_reason
+float|None motion_score
+float display_duration_sec
+str display_position
+str is_active

1.2 ad-batch — 모듈 의존 관계

```
classDiagram
    class AdBatchMain {
        +_current_week_start() str
        +_restore_previous_free_vod(conn, week_start) None
        +select_weekly_free_vod() List_Dict
        +run_fast_ad_pipeline(vod_dict) None
        +weekly_batch_job() None
    }

    class SeasonalThemes {
        +dict SEASONAL_THEMES$
        +dict SLOT_REASON_MAP$
        +build_seasonal_case_when() str$
    }

    class SceneDetector {
        +detect_scenes(video_path, work_dir) List_SceneSegment
    }

    class SceneSegment {
        +int scene_no
        +float start_time
        +float end_time
        +str|None keyframe_path
    }

    class VisionAnalyzer {
        +analyze_keyframe(keyframe_path) Dict
        -_run_yolo(img_path) List_str
        -_run_clip(img_path) List_str
        -_extract_colors(img_path) List_str
    }

    class AdGenerator {
        +generate_image_ad(keywords) Dict
        +generate_video_ad(keywords) Dict
        -_keywords_to_prompt(keywords) str
        -_dalle3_generate(prompt) Dict
        -_pil_placeholder(keywords) Dict
    }

    class TimestampCalculator {
        +calculate_insertion_points(scenes) List_Dict
        -_calc_motion_score(scene, max_dur) float
    }

    AdBatchMain --> SeasonalThemes : build_seasonal_case_when()
    AdBatchMain --> SceneDetector : detect_scenes()
    AdBatchMain --> VisionAnalyzer : analyze_keyframe()
```

```
    AdBatchMain --> AdGenerator : generate_image_ad() + generate_video_ad()
    AdBatchMain --> TimestampCalculator : calculate_insertion_points()
    SceneDetector --> SceneSegment : creates
```



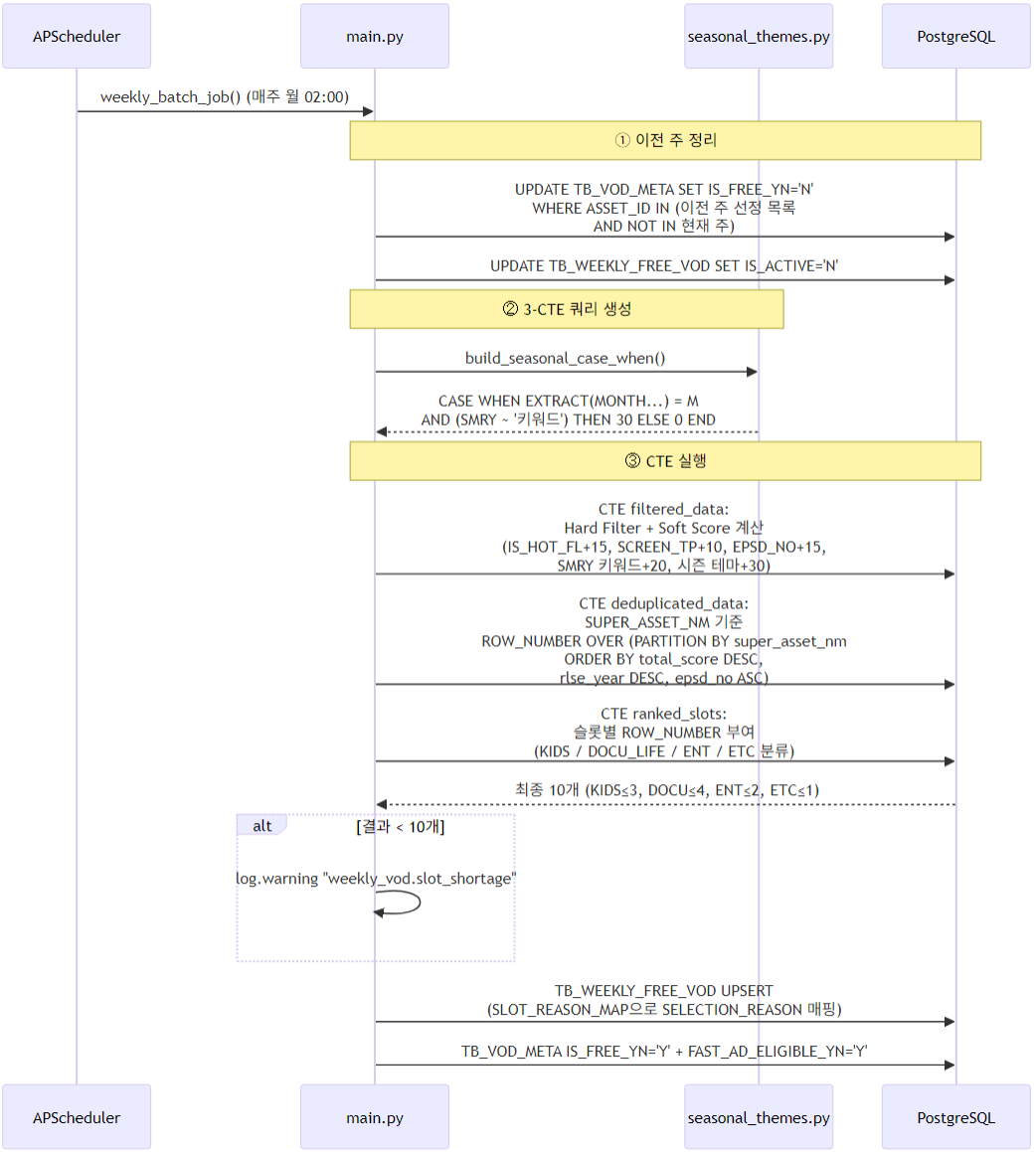
---

2. 시퀀스 다이어그램

2.1 트랙1 VOD 배치 선정 시퀀스 (v2 CTE 쿼리)

```
sequenceDiagram
    participant CRON as APScheduler
    participant MAIN as main.py
    participant THEMES as seasonal_themes.py
    participant DB as PostgreSQL

    CRON->>MAIN: weekly_batch_job() (매주 월 02:00)

    Note over MAIN,DB: ① 이전 주 정리
    MAIN->>DB: UPDATE TB_VOD_META SET IS_FREE_YN='N'<br/>WHERE ASSET_ID IN (이전 주 선정 목록<br/>AND NOT IN 현재 주)
    MAIN->>DB: UPDATE TB_WEEKLY_FREE_VOD SET IS_ACTIVE='N'

    Note over MAIN,THEMES: ② 3-CTE 쿼리 생성
    MAIN->>THEMES: build_seasonal_case_when()
    THEMES-->>MAIN: CASE WHEN EXTRACT(MONTH...) = M<br/>AND (SMRY ~ '키워드') THEN 30 ELSE 0 END

    Note over MAIN,DB: ③ CTE 실행
    MAIN->>DB: CTE filtered_data:<br/>  Hard Filter + Soft Score 계산<br/>  (IS_HOT_FL+15, SCREEN_TP+10, EPSD_NO+15,<br/>  SMRY 키워드+20, 시즌 테마+30)

    MAIN->>DB: CTE deduplicated_data:<br/>  SUPER_ASSET_NM 기준<br/>  ROW_NUMBER OVER (PARTITION BY super_asset_nm<br/>  ORDER BY total_score DESC,<br/>  rlse_year DESC, epsd_no ASC)

    MAIN->>DB: CTE ranked_slots:<br/>  슬롯별 ROW_NUMBER 부여<br/>  (KIDS / DOCU_LIFE / ENT / ETC 분류)
```

```
    DB-->>MAIN: 최종 10개 (KIDS≤3, DOCU≤4, ENT≤2, ETC≤1)

    alt 결과 < 10개
        MAIN->>MAIN: log.warning "weekly_vod.slot_shortage"
    end

    MAIN->>DB: TB_WEEKLY_FREE_VOD UPSERT<br/>(SLOT_REASON_MAP으로 SELECTION_REASON 매핑)
    MAIN->>DB: TB_VOD_META IS_FREE_YN='Y' + FAST_AD_ELIGIBLE_YN='Y'
```

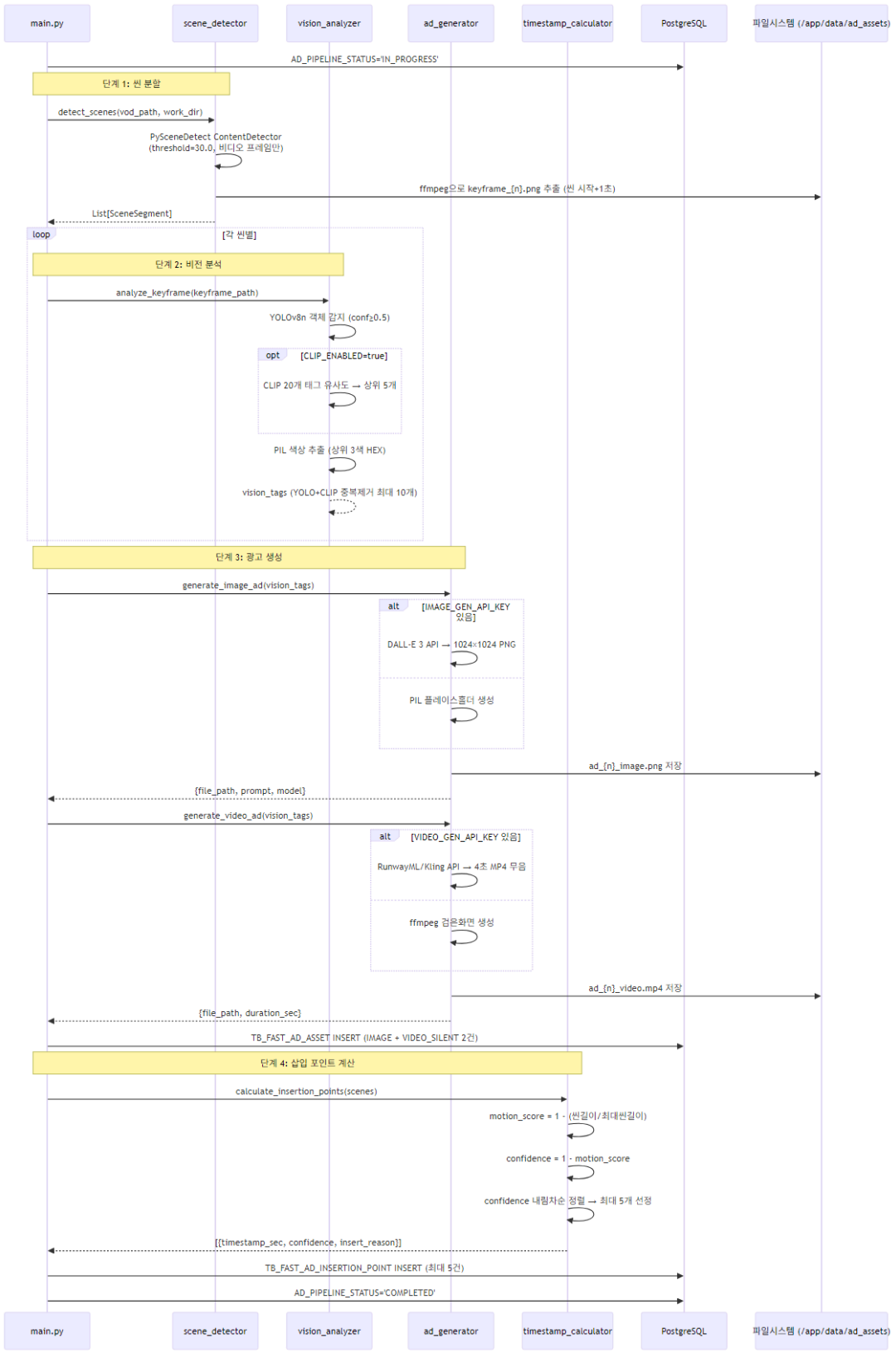


2.2 FAST 광고 파이프라인 시퀀스 (단일 VOD)

```
sequenceDiagram
    participant MAIN as main.py
    participant SCENE as scene_detector
```

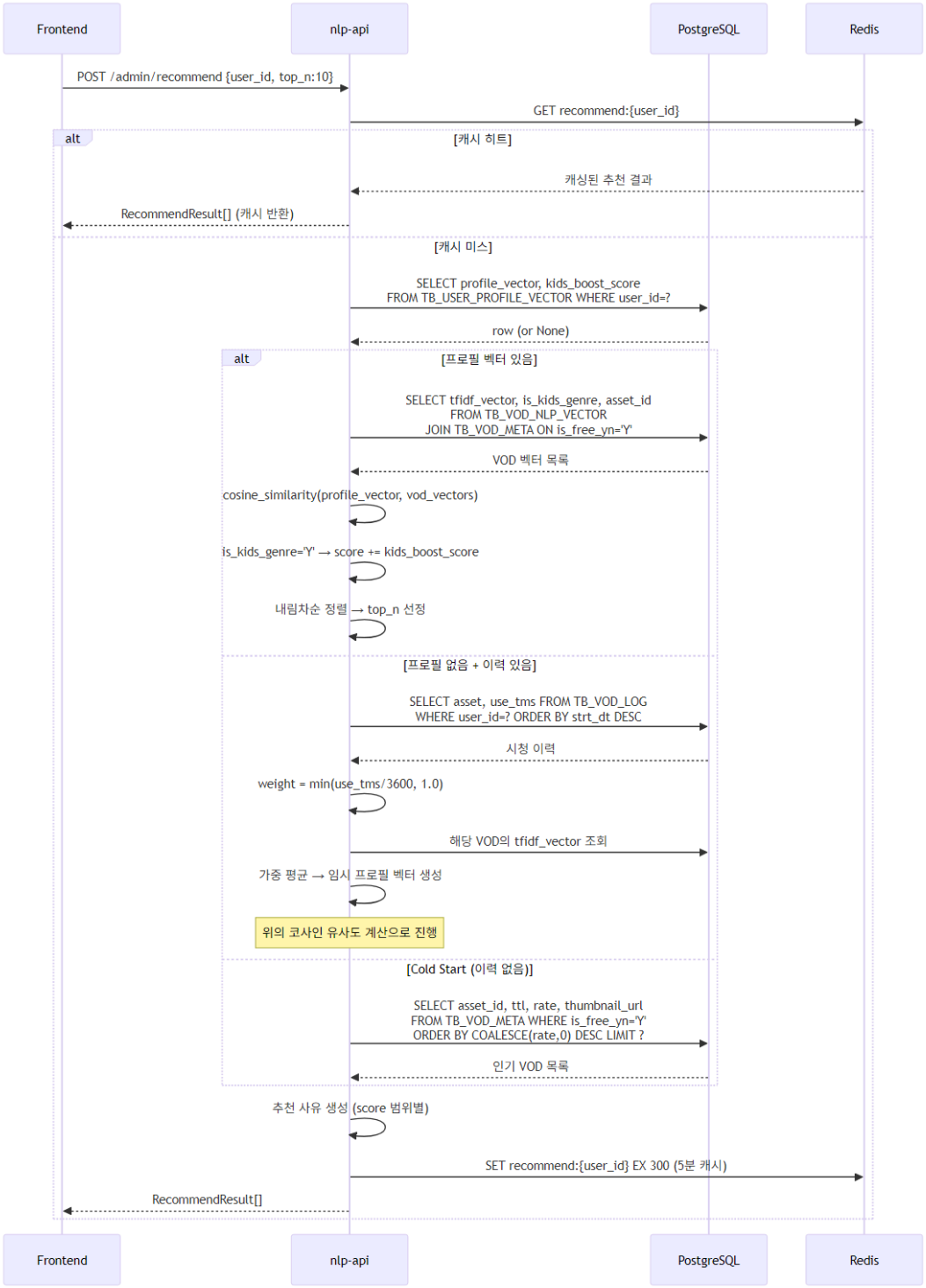
```
participant VISION as vision_analyzer
participant GEN as ad_generator
participant CALC as timestamp_calculator
participant DB as PostgreSQL
participant FS as 파일시스템 (/app/data/ad_assets)

MAIN->>DB: AD_PIPELINE_STATUS='IN_PROGRESS'

Note over MAIN,SCENE: 단계 1: 씬 분할
MAIN->>SCENE: detect_scenes(vod_path, work_dir)
SCENE->>SCENE: PySceneDetect ContentDetector<br/>(threshold=30.0, 비디오 프레임만)
SCENE->>FS: ffmpeg으로 keyframe_{n}.png 추출 (씬 시작+1초)
SCENE-->>MAIN: List[SceneSegment]

loop 각 씬별
    Note over MAIN,VISION: 단계 2: 비전 분석
    MAIN->>VISION: analyze_keyframe(keyframe_path)
    VISION->>VISION: YOLOv8n 객체 감지 (conf≥0.5)
    opt CLIP_ENABLED=true
        VISION->>VISION: CLIP 20개 태그 유사도 → 상위 5개
    end
    VISION->>VISION: PIL 색상 추출 (상위 3색 HEX)
    VISION-->>VISION: vision_tags (YOLO+CLIP 중복제거 최대 10개)
end

Note over MAIN,GEN: 단계 3: 광고 생성
MAIN->>GEN: generate_image_ad(vision_tags)
alt IMAGE_GEN_API_KEY 있음
    GEN->>GEN: DALL-E 3 API → 1024×1024 PNG
else
    GEN->>GEN: PIL 플레이스홀더 생성
end
GEN->>FS: ad_{n}_image.png 저장
GEN-->>MAIN: {file_path, prompt, model}

MAIN->>GEN: generate_video_ad(vision_tags)
alt VIDEO_GEN_API_KEY 있음
    GEN->>GEN: RunwayML/Kling API → 4초 MP4 무음
else
    GEN->>GEN: ffmpeg 검은화면 생성
end
GEN->>FS: ad_{n}_video.mp4 저장
GEN-->>MAIN: {file_path, duration_sec}

MAIN->>DB: TB_FAST_AD_ASSET INSERT (IMAGE + VIDEO_SILENT 2건)

Note over MAIN,CALC: 단계 4: 삽입 포인트 계산
MAIN->>CALC: calculate_insertion_points(scenes)
CALC->>CALC: motion_score = 1 - (씬길이/최대씬길이)
CALC->>CALC: confidence = 1 - motion_score
CALC->>CALC: confidence 내림차순 정렬 → 최대 5개 선정
CALC-->>MAIN: [{timestamp_sec, confidence, insert_reason}]
```

```
MAIN->>DB: TB_FAST_AD_INSERTION_POINT INSERT (최대 5건)
MAIN->>DB: AD_PIPELINE_STATUS='COMPLETED'
```


main.py
scene_detector
vision_analyzer
ad_generator
timestamp_calculator
PostgreSQL
파일시스템 (/app/data/ad_assets)
AD_PIPELINE_STATUS='IN_PROGRESS'
단계 1: 씬 분할
detect_scenes(vod_path, work_dir)
PySceneDetect ContentDetector
(threshold=30.0, 비디오 프레임만)
ffmpeg으로 keyframe_{n}.png 추출 (씬 시작+1초)
List[SceneSegment]
loop
[각 씬별]
단계 2: 비전 분석
analyze_keyframe(keyframe_path)
YOLOv8n 객체 감지 (conf≥0.5)
opt
[CLIP_ENABLED=true]
CLIP 20개 태그 유사도 → 상위 5개
PIL 색상 추출 (상위 3색 HEX)
vision_tags (YOLO+CLIP 중복제거 최대 10개)
단계 3: 광고 생성
generate_image_ad(vision_tags)
alt
[IMAGE_GEN_API_KEY 있음]
DALL-E 3 API → 1024×1024 PNG
PIL 플레이스홀더 생성
ad_{n}_image.png 저장
{file_path, prompt, model}
generate_video_ad(vision_tags)
alt
[VIDEO_GEN_API_KEY 있음]
RunwayML/Kling API → 4초 MP4 무음
ffmpeg 검은화면 생성
ad_{n}_video.mp4 저장
{file_path, duration_sec}
TB_FAST_AD_ASSET INSERT (IMAGE + VIDEO_SILENT 2건)
단계 4: 삽입 포인트 계산
calculate_insertion_points(scenes)
motion_score = 1 - (씬길이/최대씬길이)
confidence = 1 - motion_score
confidence 내림차순 정렬 → 최대 5개 선정
[{timestamp_sec, confidence, insert_reason}]
TB_FAST_AD_INSERTION_POINT INSERT (최대 5건)
AD_PIPELINE_STATUS='COMPLETED'
main.py
scene_detector
vision_analyzer
ad_generator
timestamp_calculator
PostgreSQL
파일시스템 (/app/data/ad_assets)

2.3 NLP 개인화 추천 시퀀스

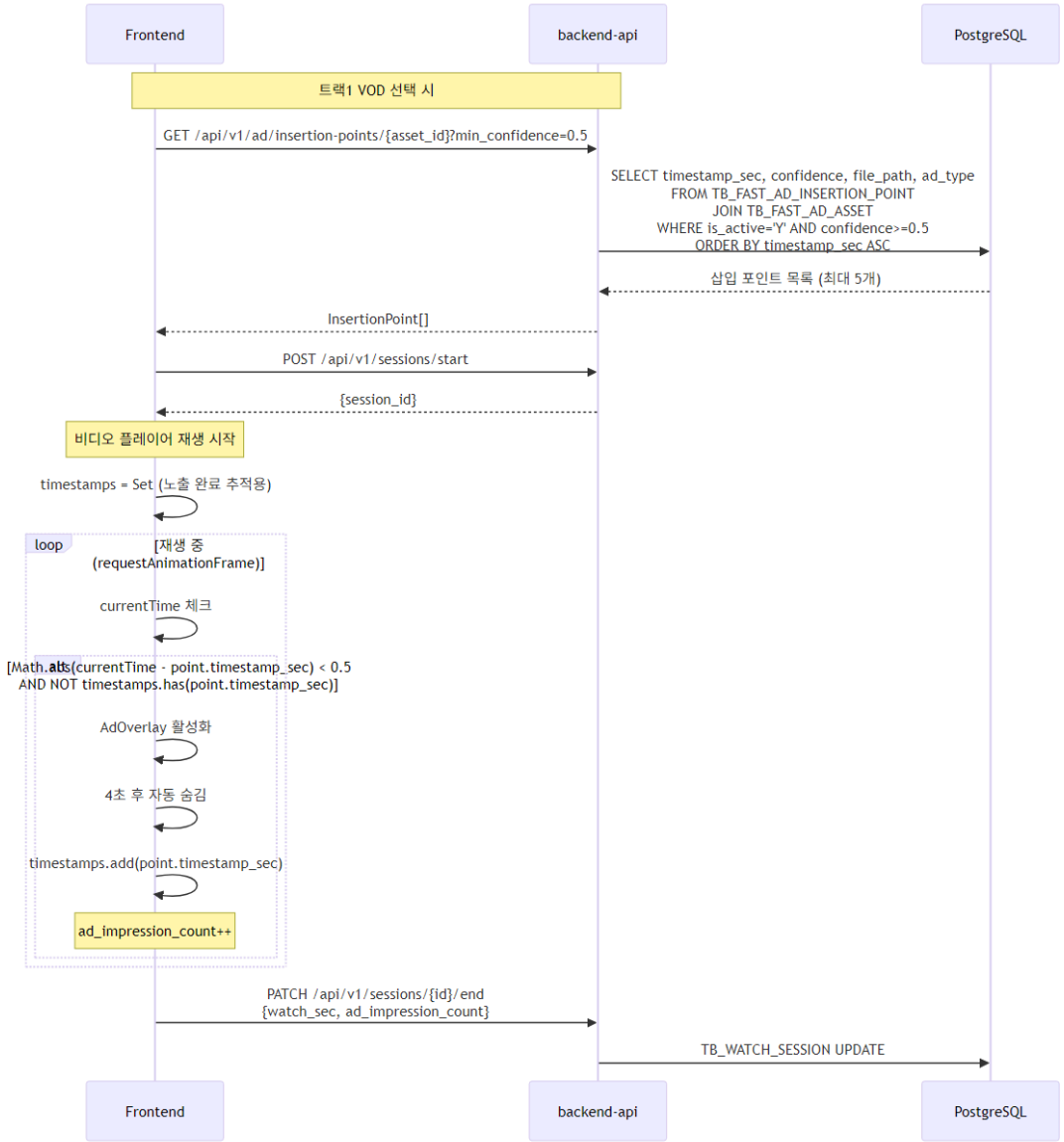
```
sequenceDiagram
    participant FE as Frontend
    participant NLP as nlp-api
    participant DB as PostgreSQL
    participant CACHE as Redis

    FE->>NLP: POST /admin/recommend {user_id, top_n:10}

    NLP->>CACHE: GET recommend:{user_id}
    alt 캐시 히트
        CACHE-->>NLP: 캐싱된 추천 결과
        NLP-->>FE: RecommendResult[] (캐시 반환)
    else 캐시 미스
        NLP->>DB: SELECT profile_vector, kids_boost_score<br/>FROM TB_USER_PROFILE_VECTOR WHERE
user_id=?
        DB-->>NLP: row (or None)

        alt 프로필 벡터 있음
            NLP->>DB: SELECT tfidf_vector, is_kids_genre, asset_id<br/>FROM
TB_VOD_NLP_VECTOR<br/>JOIN TB_VOD_META ON is_free_yn='Y'
            DB-->>NLP: VOD 벡터 목록
            NLP->>NLP: cosine_similarity(profile_vector, vod_vectors)
            NLP->>NLP: is_kids_genre='Y' → score += kids_boost_score
            NLP->>NLP: 내림차순 정렬 → top_n 선정

        else 프로필 없음 + 이력 있음
            NLP->>DB: SELECT asset, use_tms FROM TB_VOD_LOG<br/>WHERE user_id=? ORDER BY strt_dt DESC
            DB-->>NLP: 시청 이력
            NLP->>NLP: weight = min(use_tms/3600, 1.0)
            NLP->>DB: 해당 VOD의 tfidf_vector 조회
            NLP->>NLP: 가중 평균 → 임시 프로필 벡터 생성
            Note over NLP: 위의 코사인 유사도 계산으로 진행

        else Cold Start (이력 없음)
            NLP->>DB: SELECT asset_id, ttl, rate, thumbnail_url<br/>FROM TB_VOD_META WHERE
is_free_yn='Y'<br/>ORDER BY COALESCE(rate,0) DESC LIMIT ?
            DB-->>NLP: 인기 VOD 목록
        end

        NLP->>NLP: 추천 사유 생성 (score 범위별)
        NLP->>CACHE: SET recommend:{user_id} EX 300 (5분 캐시)
        NLP-->>FE: RecommendResult[]
    end
```



2.4 FAST 광고 오버레이 재생 시퀀스

```
sequenceDiagram
    participant FE as Frontend
    participant BE as backend-api
```

```
    participant DB as PostgreSQL

    Note over FE,BE: 트랙1 VOD 선택 시
    FE->>BE: GET /api/v1/ad/insertion-points/{asset_id}?min_confidence=0.5
    BE->>DB: SELECT timestamp_sec, confidence, file_path, ad_type<br/>FROM
TB_FAST_AD_INSERTION_POINT<br/>JOIN TB_FAST_AD_ASSET<br/>WHERE is_active='Y' AND
confidence>=0.5<br/>ORDER BY timestamp_sec ASC
    DB-->>BE: 삽입 포인트 목록 (최대 5개)
    BE-->>FE: InsertionPoint[]

    FE->>BE: POST /api/v1/sessions/start
    BE-->>FE: {session_id}

    Note over FE: 비디오 플레이어 재생 시작
    FE->>FE: timestamps = Set (노출 완료 추적용)

    loop 재생 중 (requestAnimationFrame)
        FE->>FE: currentTime 체크
        alt Math.abs(currentTime - point.timestamp_sec) < 0.5<br/>AND NOT
timestamps.has(point.timestamp_sec)
            FE->>FE: AdOverlay 활성화
            FE->>FE: 4초 후 자동 숨김
            FE->>FE: timestamps.add(point.timestamp_sec)
            Note over FE: ad_impression_count++
        end
    end

    FE->>BE: PATCH /api/v1/sessions/{id}/end<br/>{watch_sec, ad_impression_count}
    BE->>DB: TB_WATCH_SESSION UPDATE
```



---

3. 핵심 알고리즘 상세

3.1 소프트 점수 계산 (v2 CTE: filtered_data)

```
/* 총점 = 인기작 + 화질 + 키즈1화 + 4060키워드 + 시즌테마 (최대 90점) */
CASE WHEN IS_HOT_FL = 1 THEN 15 ELSE 0 END        -- 인기작 +15
+ CASE WHEN SCREEN_TP IN ('HD','FHD','UHD') THEN 10 ELSE 0 END   -- 고화질 +10
+ CASE WHEN (GENRE LIKE '%키즈%' OR GENRE LIKE '%애니%')
          AND EPSD_NO = 1 THEN 15 ELSE 0 END        -- 키즈 1화 유도 +15
+ CASE WHEN SMRY ~ '(건강|자연인|고향|밥상|다큐|트로트)' THEN 20 ELSE 0 END  -- 4060 +20
+ [seasonal_themes.build_seasonal_case_when()]      -- 시즌 테마 +30

AS total_score
```

점수 분포 예시:

장르	IS_HOT	SCREEN_TP	EPSD_NO	4060	시즌	합계
키즈 1화 (봄)	✅	FHD	1	-	봄 매칭	70점
다큐 인기	✅	UHD	-	✅	-	45점
예능 일반	-	HD	-	-	-	10점

3.2 시리즈 중복 제거 (v2 CTE: deduplicated_data)

```
ROW_NUMBER() OVER (
    PARTITION BY COALESCE(SUPER_ASSET_NM, TTL)   -- 시리즈명이 없으면 제목으로 그룹화
    ORDER BY
        total_score DESC,     -- ① 최고점 VOD 우선
        RLSE_YEAR DESC,       -- ② 동점 시 최신 연도
        EPSD_NO ASC           -- ③ 같은 연도이면 1화 우선
) AS dedup_rank
-- WHERE dedup_rank = 1  (각 시리즈당 1편만 선택)
```

3.3 슬롯 배분 (v2 CTE: ranked_slots)

```
CASE
    WHEN GENRE LIKE '%키즈%' OR GENRE LIKE '%애니%'
         OR GENRE LIKE '%만화%' THEN 'KIDS'
    WHEN GENRE LIKE '%다큐%' OR GENRE LIKE '%교양%'
         OR GENRE LIKE '%자연%' THEN 'DOCU_LIFE'
    WHEN GENRE LIKE '%예능%' OR GENRE LIKE '%연예%'
         OR GENRE LIKE '%음악%' THEN 'ENTERTAINMENT'
    ELSE 'ETC'
END AS slot_group,

ROW_NUMBER() OVER (
    PARTITION BY slot_group
    ORDER BY total_score DESC
) AS slot_rank

-- 최종 필터:
-- slot_group='KIDS' AND slot_rank <= 3
-- slot_group='DOCU_LIFE' AND slot_rank <= 4
-- slot_group='ENTERTAINMENT' AND slot_rank <= 2
-- slot_group='ETC' AND slot_rank <= 1
```

3.4 광고 삽입 포인트 계산 (motion_score)

```
def calculate_insertion_points(scenes: List[SceneSegment], max_count: int = 5):
    if not scenes:
```

```
        return []

    max_duration = max(s.end_time - s.start_time for s in scenes)

    candidates = []
    for scene in scenes:
        duration = scene.end_time - scene.start_time
        motion_score = 1.0 - (duration / max_duration)   # 짧을수록 고움직임
        confidence = 1.0 - motion_score                   # 긴 씬 = 저움직임 = 적합

        if confidence >= 0.5:                             # 최소 신뢰도 0.5
            if confidence >= 0.8:
                reason = "LOW_MOTION"
            elif confidence >= 0.6:
                reason = "SCENE_BREAK"
            else:
                reason = "QUIET_MOMENT"

            candidates.append({
                "timestamp_sec": scene.start_time + 1.0,  # 씬 시작 +1초
                "confidence": confidence,
                "insert_reason": reason,
                "motion_score": motion_score,
            })

    # confidence 내림차순 정렬 → 최대 5개 반환
    return sorted(candidates, key=lambda x: -x["confidence"])[:max_count]
```

3.5 유저 프로필 벡터 계산

```
def build_user_profile(vod_log_rows, nlp_vectors):
    """
    vod_log_rows: [(asset_id, use_tms), ...]
    nlp_vectors:  {asset_id: tfidf_vector}
    """
    weighted_sum = None
    total_weight = 0.0
    kids_watch_sec = 0.0
    total_watch_sec = 0.0

    for asset_id, use_tms in vod_log_rows:
        weight = min(use_tms / 3600, 1.0)   # 최대 1시간 = 1.0
        vec = nlp_vectors.get(asset_id)
        if vec is None:
            continue

        if weighted_sum is None:
            weighted_sum = [v * weight for v in vec]
        else:
            for i, v in enumerate(vec):
```

```
            weighted_sum[i] += v * weight

        total_weight += weight
        total_watch_sec += use_tms
        if is_kids_genre(asset_id):        # TB_VOD_NLP_VECTOR.IS_KIDS_GENRE
            kids_watch_sec += use_tms

    if total_weight == 0:
        return None  # Cold Start fallback

    profile_vector = [v / total_weight for v in weighted_sum]

    kids_ratio = kids_watch_sec / max(total_watch_sec, 1)
    kids_boost = max(0.1, min(1.0, 0.3 + kids_ratio * 0.7))

    return {
        "profile_vector": profile_vector,
        "kids_boost_score": kids_boost,
        "total_watch_sec": total_watch_sec,
    }
```

---

4. 에러 처리 전략 (VOD 범위)

에러 유형	처리 방법	로그 레벨
VOD 파일 없음	더미 씬 1개 생성 → 파이프라인 계속	WARNING
AI API 키 없음	PIL 플레이스홀더 / ffmpeg 검은화면	WARNING
CLIP 로드 실패	YOLO만으로 계속(graceful fallback)	WARNING
슬롯 10개 미달	`weekly_vod.slot_shortage` 경고 + 선정된 수만 기록	WARNING
단일 VOD 파이프라인 실패	`FAILED` 기록 + 다음 VOD 계속	ERROR
tfidf.pkl 없음	`tfidf_ready=false` 경고, Cold Start로 fallback	WARNING
DB 연결 실패	pool_pre_ping=True 자동 재연결	ERROR
추천 API 실패	프론트엔드 오류 표시 → 사용자 재시도 안내	ERROR

---

5. 환경별 동작 행렬

항목	개발 (CPU)	스테이징 (GPU)	운영 (GPU)
CLIP	`false`	`true`	`true`
이미지 광고	PIL 플레이스홀더	테스트 API	DALL-E 3 실제
영상 광고	ffmpeg 검은화면	테스트 API	RunwayML 실제
VOD 파일	더미 씬	일부 존재	전체 존재

TF-IDF 모델	수동 실행 필요	미리 실행	미리 실행